Part 7

Modules & Advanced Terraform

Professional Terraform Notes · Beginner to Advanced · English Technical Notes with Simple Kannada Explanations

These notes cover reusable Terraform modules, workspaces, dynamic blocks, for_each, count, and advanced Terraform patterns. Each topic follows your five-heading format and includes practical AWS project examples and scenario-based interview questions with answers.

Topics covered

1. Terraform Modules
2. Reusable Modules and Module Structure
3. Module Inputs, Outputs, and Versioning
4. Terraform Workspaces
5. Dynamic Blocks
6. for_each
7. count
8. for_each vs count
9. Conditional Expressions and for Expressions
10. depends_on, lifecycle, and Advanced Resource Management
11. Module Composition and Environment Design
12. Advanced Terraform Best Practices and Production Scenarios

1. Terraform Modules

1.1 Definition — What is a Terraform Module?

A Terraform module is a collection of Terraform configuration files grouped together to manage related infrastructure resources.

A directory containing one or more .tf files is a Terraform module. The directory from which you run Terraform commands is called the root module. Modules called by the root module are called child modules.

For example, an EC2 module might contain the configuration required to create an EC2 instance, while a security group module manages inbound and outbound network rules.

> **Kannada explanation:** Terraform Module ಅಂದ್ರೆ, ಒಂದೇ ರೀತಿಯ infrastructure resourcesಗಳನ್ನು ಒಂದು folder ಒಳಗೆ organize ಮಾಡಿ ಇಡುವುದು. ಉದಾಹರಣೆಗೆ EC2 create ಮಾಡುವ code ಅನ್ನು ಪ್ರತ್ಯೇಕ module ಆಗಿ ಇಟ್ಟರೆ, ಬೇರೆ projectಗಳಲ್ಲೂ ಅದೇ code ಅನ್ನು ಮತ್ತೆ ಬಳಸಬಹುದು.

1.2 Purpose — Why do we use modules?

- Reusability: Reuse the same infrastructure code across multiple projects.
- Maintainability: Keep large Terraform configurations organized.

- Consistency: Apply standard configurations across environments.
- Scalability: Divide a large infrastructure project into smaller components.
- Team collaboration: Different team members can maintain separate modules.
- Reduced duplication: Avoid copying the same resource definitions into multiple directories.

> **Kannada explanation:** ಪ್ರತಿ projectನಲ್ಲೂ EC2, security group, IAM roleಗೆ ಮತ್ತೆ ಮತ್ತೆ code ಬರೆಯುವ ಬದಲು module ಮಾಡಿಟ್ಟುಕೊಂಡರೆ ಅದನ್ನೇ reuse ಮಾಡಬಹುದು. ಇದರಿಂದ duplicate code ಕಡಿಮೆಯಾಗುತ್ತದೆ ಮತ್ತು maintenance ಸುಲಭವಾಗುತ್ತದೆ.

1.3 Working — How does a module work?

```
Root Module
  main.tf · variables.tf · outputs.tf
        |
        |  Calls a child module and passes input values
        v
EC2 Child Module
  Creates EC2 instances using its own configuration
        |
        |  Returns outputs to the root module
        v
AWS Infrastructure
  EC2 instance · instance ID · public IP
```

The workflow is:

1. Create a module directory.
2. Define resources inside the module.
3. Declare module input variables.
4. Call the module from the root configuration using a module block.
5. Pass required inputs to the module.
6. Expose selected values through module outputs.
7. Run terraform init, terraform plan, and terraform apply.

> **Kannada explanation:** Root module ಮುಖ್ಯ configuration ಆಗಿರುತ್ತದೆ. ಅದು child moduleಗೆ ಬೇಕಾದ values ಕೊಡುತ್ತದೆ. Child module resources create ಮಾಡುತ್ತದೆ. ಬೇಕಾದ output values ಅನ್ನು root moduleಗೆ ಹಿಂದಿರುಗಿಸುತ್ತದೆ.

1.4 Practical Example — AWS EC2 module

Example project structure:

```
terraform-project/
├── main.tf
├── variables.tf
├── outputs.tf
└── modules/
    └── ec2/
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```

Root module — main.tf

```
module "web_server" {
  source        = "./modules/ec2"
  ami_id        = var.ami_id
  instance_type = "t3.micro"
  subnet_id     = var.subnet_id
}
```

Child module — modules/ec2/variables.tf

```
variable "ami_id" {
  type = string
}

variable "instance_type" {
  type    = string
  default = "t3.micro"
}

variable "subnet_id" {
  type = string
}
```

Child module — modules/ec2/main.tf

```
resource "aws_instance" "this" {
  ami           = var.ami_id
  instance_type = var.instance_type
  subnet_id     = var.subnet_id
}
```

Child module — modules/ec2/outputs.tf

```
output "instance_id" {
  value = aws_instance.this.id
}
```

Root module — outputs.tf

```
output "web_server_id" {
  value = module.web_server.instance_id
}
```

> **Kannada explanation:** source module ಇರುವ folder ಅನ್ನು ಸೂಚಿಸುತ್ತದೆ. Root module AMI, instance type, subnet ID values ಕೊಡುತ್ತದೆ. EC2 module instance create ಮಾಡುತ್ತದೆ. instance_id output ಮೂಲಕ ಅದರ ID ಅನ್ನು ಹೊರಗೆ ಪಡೆಯಬಹುದು.

1.5 Scenario-based Interview Questions

Q1. Your organization creates EC2 instances with the same security standards across 10 projects. How would you avoid duplicate code?

Answer: Create a reusable EC2 module that accepts inputs such as AMI ID, instance type, subnet ID, and security group IDs. Call the module from each project with the required values.

> **Kannada:** ಹತ್ತು projectಗಳಲ್ಲೂ ಒಂದೇ EC2 code ಬರೆಯುವ ಬದಲು common module ಮಾಡಿ ಎಲ್ಲ projectಗಳಲ್ಲೂ use ಮಾಡುತ್ತೇನೆ.

Q2. What is the difference between a root module and a child module?

Answer: The root module is the working directory where Terraform commands are executed. A child module is another module invoked by the root or another module.

Q3. Does creating a module automatically create AWS resources?

Answer: No. The module must be called by an active configuration, and its resources must be applied through Terraform.

Q4. Can a module create multiple resources?

Answer: Yes. A module can contain multiple resources, data sources, variables, outputs, and even calls to other modules.

2. Reusable Modules and Module Structure

2.1 Definition — What are reusable modules?

A reusable module is a Terraform module designed to be used by multiple projects or environments through configurable input variables.

Instead of hardcoding every value, the module accepts parameters such as region, instance type, VPC ID, subnet ID, and tags.

A well-designed module typically contains:

- main.tf — resources and module logic.
- variables.tf — input variables.
- outputs.tf — output values.
- versions.tf — Terraform and provider requirements.
- README.md — usage instructions and examples.

> **Kannada explanation:** Reusable module ಅಂದ್ರೆ, ಒಂದು projectಗೆ ಮಾತ್ರ ಸೀಮಿತವಾಗಿರದೆ ಬೇರೆ projectಗಳಲ್ಲೂ ಬಳಸುವಂತೆ ಮಾಡಿರುವ module. Hardcode ಮಾಡುವ ಬದಲು variables ಬಳಸುವುದರಿಂದ ಬೇಕಾದ values ಬದಲಾಯಿಸಿ ಅದೇ module ಬಳಸಬಹುದು.

2.2 Purpose — Why do we use reusable modules?

- Standardize infrastructure across development, testing, and production.
- Reduce repeated code.
- Simplify code reviews.
- Centralize infrastructure improvements.
- Make infrastructure easier to test and maintain.

> **Kannada explanation:** ಎಲ್ಲ environmentಗಳಲ್ಲೂ ಒಂದೇ standard follow ಮಾಡಲು reusable modules ಸಹಾಯ ಮಾಡುತ್ತವೆ. Moduleನಲ್ಲಿ security improvement ಮಾಡಿದರೆ ಅದನ್ನು ಬಳಸುವ projectsಗಳಿಗೆ controlled ರೀತಿಯಲ್ಲಿ update ಮಾಡಬಹುದು.

2.3 Working — How does it work?

A module defines the infrastructure logic once. Each caller supplies its own input values.

For example:

- Development: t3.micro
- Testing: t3.small
- Production: t3.medium

The same module can create different infrastructure configurations based on those inputs.

> **Kannada explanation:** Module code ಒಂದೇ ಇರುತ್ತದೆ. Devಗೆ ಒಂದು instance type, productionಗೆ ಬೇರೆ instance type ಕೊಡಬಹುದು. Values ಬದಲಾಗುತ್ತವೆ; common infrastructure logic ಒಂದೇ ಆಗಿರುತ್ತದೆ.

2.4 Practical Example — Reuse a module across environments

```
module "dev_server" {
  source        = "./modules/ec2"
  ami_id        = var.ami_id
  instance_type = "t3.micro"
  subnet_id     = var.dev_subnet_id
}

module "test_server" {
  source        = "./modules/ec2"
  ami_id        = var.ami_id
  instance_type = "t3.small"
  subnet_id     = var.test_subnet_id
}
```

Both module calls use the same EC2 module but pass different values.

For production, use a separate module call or environment configuration with production-specific settings and access controls.

2.5 Scenario-based Interview Questions

Q1. How would you design reusable modules for a large AWS project?

Answer: Create separate modules for networking, EC2, IAM, security groups, EKS, and monitoring. Define clear inputs and outputs, document each module, and version shared modules.

Q2. A security fix is required in a module used by 20 projects. What would you do?

Answer: Update and test the module, release a new version, then upgrade consumers in a controlled manner. Review Terraform plans before applying the changes.

Q3. Should every Terraform resource have its own module?

Answer: Not necessarily. Modules should represent useful, reusable infrastructure components. Excessively small modules can add complexity without meaningful reuse.

3. Module Inputs, Outputs, and Versioning

3.1 Definition — What are module inputs, outputs, and versioning?

- Inputs: Values passed into a module through variables.
- Outputs: Values exposed by a module for use by its caller.
- Versioning: Identifying and controlling which version of a reusable module is used.

Versioning is particularly important when modules are stored in Git repositories or a Terraform module registry.

> **Kannada explanation:** Inputs ಅಂದ್ರೆ moduleಗೆ ಕೊಡುವ values. Outputs ಅಂದ್ರೆ moduleನಿಂದ ಹೊರಗೆ ಪಡೆಯುವ values. Versioning ಅಂದ್ರೆ ಯಾವ module version ಬಳಸುತ್ತಿದ್ದೇವೆ ಅನ್ನೋದನ್ನು control ಮಾಡುವುದು.

3.2 Purpose — Why do we use them?

Inputs make modules configurable. Outputs allow modules to share useful information, such as VPC IDs or EC2 instance

IDs. Versioning helps teams avoid unexpected changes when a shared module is updated.

> **Kannada explanation:** Moduleಗೆ ಬೇಕಾದ values inputs ಮೂಲಕ ಕೊಡುತ್ತೇವೆ. Create ಆದ resourceನ IDಯಂತಹ ಮಾಹಿತಿ outputs ಮೂಲಕ ಪಡೆಯುತ್ತೇವೆ. Version pin ಮಾಡಿದರೆ module update ಆದ ತಕ್ಷಣ ನಮ್ಮ projectನಲ್ಲಿ ಅನಿರೀಕ್ಷಿತ ಬದಲಾವಣೆ ಆಗುವುದನ್ನು ತಪ್ಪಿಸಬಹುದು.

3.3 Working — How do they work?

1. Declare variables in the child module.
2. Pass values in the module call.
3. Define outputs in the child module.
4. Read child module outputs using module.<module_name>.<output_name>.
5. Pin a Git module to a tested release or commit when appropriate.

3.4 Practical Example — Module input, output, and version

```
module "network" {
  source = "git::https://github.com/example/terraform-aws-network.git?ref=v1.2.0"

  vpc_cidr = "10.20.0.0/16"
}
```

Inside the module, variables.tf:

```
variable "vpc_cidr" {
  type        = string
  description = "CIDR range for the VPC"
}
```

Inside the module, outputs.tf:

```
output "vpc_id" {
  value = aws_vpc.this.id
}
```

The root module can use the output:

```
output "network_vpc_id" {
  value = module.network.vpc_id
}
```

Important: The repository URL and module resource names above are illustrative. The source repository must exist and contain a compatible module.

> **Kannada explanation:** ಇಲ್ಲಿ ref=v1.2.0 ಅಂದರೆ Git repositoryಯಲ್ಲಿರುವ ನಿರ್ದಿಷ್ಟ version ಬಳಸುವುದು. Module create ಮಾಡಿದ VPC ID ಅನ್ನು module.network.vpc_id ಮೂಲಕ ಪಡೆಯುತ್ತೇವೆ.

3.5 Scenario-based Interview Questions

Q1. A shared module has a breaking change. How do you prevent it from unexpectedly affecting production?

Answer: Pin the module to a tested version, test the newer version in a lower environment, review the plan, and upgrade production in a controlled release.

Q2. Can a root module directly access a resource inside a child module?

Answer: The recommended interface is to expose the required value through a child module output and reference it through the module call.

Q3. Does pinning a module version guarantee the whole deployment is reproducible?

Answer: Not by itself. Provider versions, Terraform versions, external dependencies, configuration values, and state must also be managed appropriately.

4. Terraform Workspaces

4.1 Definition — What are Terraform workspaces?

Terraform CLI workspaces allow one Terraform configuration and backend to manage multiple separate state instances.

The default workspace is named default. Additional workspaces can be created using commands such as terraform workspace new.

> **Kannada explanation:** Workspaces ಅಂದ್ರೆ ಒಂದೇ Terraform configurationಗೆ ಬೇರೆ ಬೇರೆ stateಗಳನ್ನು maintain ಮಾಡುವುದು. ಉದಾಹರಣೆಗೆ dev ಮತ್ತು test workspaces ಇದ್ದರೆ ಅವುಗಳ Terraform state ಪ್ರತ್ಯೇಕವಾಗಿರಬಹುದು.

4.2 Purpose — Why do we use workspaces?

- Maintain separate state for multiple instances of the same configuration.
- Support simple environment variations.
- Experiment without changing the default workspace's state.
- Reduce duplication when environments genuinely share the same infrastructure design.

> **Kannada explanation:** ಒಂದೇ configuration ಬಳಸಿ ಬೇರೆ environmentಗಳಿಗೆ ಪ್ರತ್ಯೇಕ state ಬೇಕಾದಾಗ workspaces ಸಹಾಯ ಮಾಡುತ್ತವೆ. ಆದರೆ production isolationಗೆ workspace ಒಂದೇ ಸಾಕಾಗುವುದಿಲ್ಲ.

4.3 Working — How do workspaces work?

Create and select workspaces:

```
terraform workspace list
terraform workspace new dev
terraform workspace new test
terraform workspace select dev
terraform workspace show
```

The selected workspace determines which state instance Terraform operates on for the current configuration and backend.

In an S3 backend using the default workspace, the configured key is used. Non-default workspaces use a workspace-specific state location based on the backend's workspace settings and key layout.

> **Kannada explanation:** workspace select dev ಮಾಡಿದರೆ Terraform dev workspaceನ state ಬಳಸುತ್ತದೆ. test ಆಯ್ಕೆ ಮಾಡಿದರೆ test state ಬಳಸುತ್ತದೆ. Command run ಮಾಡುವ ಮೊದಲು ಯಾವ workspace select ಆಗಿದೆ ಎಂದು check ಮಾಡಬೇಕು.

4.4 Practical Example — Workspace-based naming

```
resource "aws_s3_bucket" "app_logs" {
  bucket = "company-app-logs-${terraform.workspace}"
}
```

If the active workspace is dev, the bucket name would be company-app-logs-dev. In test, it would be company-app-logs-test.

This is an illustrative naming pattern; S3 bucket names must be globally unique.

Production warning: CLI workspaces are not a strong security boundary. For critical production environments, separate root configurations, state locations, AWS accounts, IAM permissions, and deployment approvals may be more appropriate.

4.5 Scenario-based Interview Questions

Q1. Your team wants separate development and production states. Would you always recommend workspaces?

Answer: No. Workspaces can be suitable for simple environments with similar configuration, but production often benefits from stronger isolation using separate root configurations, state backends or keys, and AWS accounts.

Q2. What happens if you switch from dev to test?

Answer: Terraform switches to the state associated with test. It does not automatically copy the dev state into test.

Q3. Do workspaces create separate AWS accounts?

Answer: No. Workspaces separate Terraform state instances; AWS account separation must be configured independently.

5. Dynamic Blocks

5.1 Definition — What is a dynamic block?

A dynamic block generates repeated nested configuration blocks based on a collection, such as a list or map.

Unlike count and for_each, which can create multiple resource instances or module instances, a dynamic block generates nested blocks inside a resource or another supported block.

Common examples include generating security group ingress rules or repeated configuration blocks for supported resources.

> **Kannada explanation:** dynamic block ಅಂದ್ರೆ resource ಒಳಗೆ ಒಂದೇ ತರಹದ nested blocks ಅನ್ನು list ಅಥವಾ map ಆಧಾರದಲ್ಲಿ automatic ಆಗಿ generate ಮಾಡುವುದು. Security groupನಲ್ಲಿ ಹಲವು ingress rules ಇದ್ದಾಗ ಪ್ರತಿ ruleಗೂ ಪ್ರತ್ಯೇಕ block ಬರೆಯುವ ಬದಲು ಬಳಸಬಹುದು.

5.2 Purpose — Why do we use dynamic blocks?

- Reduce repeated nested configuration.
- Build configuration from lists or maps.
- Simplify resources that contain many similar nested blocks.
- Make reusable modules more flexible.

> **Kannada explanation:** ಹತ್ತು security group rules ಇದ್ದರೆ ಹತ್ತು ಬಾರಿ ಅದೇ ರೀತಿಯ code ಬರೆಯಬೇಕಾಗುವುದಿಲ್ಲ. Rules listನಲ್ಲಿ ಇಟ್ಟು dynamic block ಮೂಲಕ generate ಮಾಡಬಹುದು.

5.3 Working — How does a dynamic block work?

A dynamic block has these main parts:

- dynamic "block_name" — specifies the nested block to generate.
- for_each — specifies the collection to iterate over.
- iterator — optionally gives the current item a custom name.
- content — defines the content of each generated block.

The nested block type must be supported by the resource or configuration block in which it is used.

5.4 Practical Example — AWS security group rules

```
variable "ingress_rules" {
  type = list(object({
    port        = number
    protocol    = string
    cidr_blocks = list(string)
  }))
}

resource "aws_security_group" "web" {
  name   = "web-security-group"
  vpc_id = var.vpc_id

  dynamic "ingress" {
    for_each = var.ingress_rules
    iterator = rule

    content {
      from_port   = rule.value.port
      to_port     = rule.value.port
      protocol    = rule.value.protocol
      cidr_blocks = rule.value.cidr_blocks
    }
  }
}
```

Example input:

```
ingress_rules = [
  {
    port        = 80
    protocol    = "tcp"
    cidr_blocks = ["10.0.0.0/16"]
  },
  {
    port        = 443
    protocol    = "tcp"
    cidr_blocks = ["10.0.0.0/16"]
  }
]
```

This generates two nested ingress blocks, one for port 80 and another for port 443.

Security note: The example limits inbound access to a private CIDR range. Use only the network ranges and ports required by your application. Do not expose administrative ports broadly.

> **Kannada explanation:** for_each ಪ್ರತಿ rule ಮೇಲೆ loop ಮಾಡುತ್ತದೆ. rule.value.port ಮೂಲಕ ಆ ruleನ port ಸಿಗುತ್ತದೆ. ಹೀಗಾಗಿ listನಲ್ಲಿ ಎರಡು rules ಇದ್ದರೆ ಎರಡು ingress blocks generate ಆಗುತ್ತವೆ.

5.5 Scenario-based Interview Questions

Q1. A security group module must accept a variable number of ingress rules. How would you implement it?

Answer: Accept a typed list or map variable and use a dynamic "ingress" block to generate the nested ingress configuration.

Q2. Can you use a dynamic block to create multiple EC2 resources?

Answer: No. A dynamic block generates nested blocks. Use a resource-level for_each or count to create multiple EC2 instances.

Q3. When should you avoid dynamic blocks?

Answer: Avoid them when the number of nested blocks is fixed and small, or when dynamic generation makes the configuration harder to read or debug.

6. for_each Meta-Argument

6.1 Definition — What is for_each?

for_each is a Terraform meta-argument that creates one resource or module instance for each element in a map or set of strings.

Each instance receives a stable key that becomes part of its Terraform address.

> **Kannada explanation:** for_each ಅಂದ್ರೆ map ಅಥವಾ string setನಲ್ಲಿ ಎಷ್ಟು items ಇವೆಯೋ ಅಷ್ಟು resource instances create ಮಾಡುವುದು. ಪ್ರತಿ instanceಗೆ ಅದರದೇ key ಇರುತ್ತದೆ.

6.2 Purpose — Why do we use for_each?

- Create multiple resources from structured input.
- Give each resource a meaningful identity.
- Manage resources using stable keys.
- Reduce duplicate resource blocks.
- Handle named infrastructure components more predictably than index-based patterns in many situations.

> **Kannada explanation:** ಒಂದೇ ರೀತಿಯ resources ಅನ್ನು ಹೆಸರುಗಳ ಆಧಾರದಲ್ಲಿ manage ಮಾಡಬಹುದು. ಉದಾಹರಣೆಗೆ dev, test, prod ಎಂಬ keyಗಳ ಮೂಲಕ ಬೇರೆ ಬೇರೆ resources create ಮಾಡಬಹುದು.

6.3 Working — How does for_each work?

Terraform evaluates the collection to determine which instances should exist. The keys of the map, or the values of a set of strings, identify instances.

For example, if the keys are dev and test, Terraform creates addresses such as:

```
aws_s3_bucket.logs["dev"]
aws_s3_bucket.logs["test"]
```

The collection's keys or set members must be known before Terraform performs remote resource operations.

6.4 Practical Example — Create multiple S3 buckets

```
variable "bucket_names" {
  type = map(string)

  default = {
    dev  = "company-dev-logs-example"
    test = "company-test-logs-example"
    prod = "company-prod-logs-example"
  }
}

resource "aws_s3_bucket" "logs" {
  for_each = var.bucket_names

  bucket = each.value

  tags = {
    Environment = each.key
    ManagedBy   = "Terraform"
  }
}
```

Here:

- each.key is dev, test, or prod.
- each.value is the corresponding bucket name.
- The map values must be valid, globally unique S3 bucket names.

Example commands:

```
terraform init
terraform validate
terraform plan
```

Review the plan before applying. For real production environments, apply only after required approvals and security checks.

> **Kannada explanation:** each.key environment ಹೆಸರನ್ನು ಕೊಡುತ್ತದೆ. each.value bucket ಹೆಸರನ್ನು ಕೊಡುತ್ತದೆ. Mapನಲ್ಲಿ ಮೂರು entries ಇರುವುದರಿಂದ ಮೂರು S3 bucket resources ನಿರೀಕ್ಷಿಸಲಾಗುತ್ತದೆ.

6.5 Scenario-based Interview Questions

Q1. You must create 50 named S3 buckets from a configuration map. Would you use for_each or duplicate 50 resource blocks?

Answer: Use for_each with a map so the configuration stays concise and each bucket has a stable key.

Q2. What happens if you remove a key from a for_each map?

Answer: Terraform plans to destroy the corresponding resource instance if it is still managed by that configuration and no other changes alter the outcome. Review the plan carefully.

Q3. Can a sensitive value be used as a for_each collection?

Answer: Sensitive values cannot be used directly as for_each collections because instance keys appear in resource addresses and Terraform output. Use non-sensitive identifiers for keys and handle sensitive values separately.

7. count Meta-Argument

7.1 Definition — What is count?

count is a Terraform meta-argument that creates a specified number of resource or module instances.

Terraform identifies each instance using a numeric index starting at zero.

> **Kannada explanation:** count ಅಂದ್ರೆ ಎಷ್ಟು resources ಬೇಕು ಎಂದು number ಕೊಟ್ಟು create ಮಾಡುವುದು. count = 3 ಅಂದರೆ ಮೂರು instances. ಅವುಗಳ index 0, 1, 2 ಆಗಿರುತ್ತದೆ.

7.2 Purpose — Why do we use count?

- Create a fixed number of similar resources.
- Create an optional resource using count = 0 or count = 1.
- Reduce duplicate configuration.
- Build simple resource groups when individual identities are not essential.

> **Kannada explanation:** ಒಂದು resource ಬೇಕಿದ್ದರೆ create ಮಾಡುವುದು, ಬೇಡದಿದ್ದರೆ skip ಮಾಡುವುದು ಅಥವಾ ನಿರ್ದಿಷ್ಟ ಸಂಖ್ಯೆಯ resources create ಮಾಡುವುದು countನ ಮುಖ್ಯ ಉಪಯೋಗ.

7.3 Working — How does count work?

Terraform creates instances with index-based addresses.

For example:

```
aws_instance.worker[0]
aws_instance.worker[1]
aws_instance.worker[2]
```

Inside the resource block, count.index identifies the current instance index.

7.4 Practical Example — Create three EC2 instances

```
variable "instance_count" {
  type    = number
  default = 3
}

resource "aws_instance" "worker" {
  count         = var.instance_count
  ami           = var.ami_id
  instance_type = "t3.micro"

  tags = {
    Name = "worker-${count.index}"
  }
}
```

If instance_count = 3, Terraform plans three instances named worker-0, worker-1, and worker-2.

Optional resource example:

```
variable "create_monitoring_server" {
  type    = bool
  default = false
}

resource "aws_instance" "monitoring" {
  count         = var.create_monitoring_server ? 1 : 0
  ami           = var.ami_id
  instance_type = "t3.micro"
}
```

With the default value false, the monitoring instance is not created by this resource block.

> **Kannada explanation:** count.index ಪ್ರತಿಯೊಂದು instanceಗೆ number ಕೊಡುತ್ತದೆ. Conditional countನಲ್ಲಿ condition true ಆದರೆ ಒಂದು instance, false ಆದರೆ ಶೂನ್ಯ instance create ಆಗುತ್ತದೆ.

7.5 Scenario-based Interview Questions

Q1. A monitoring EC2 instance is required only in production. How would you manage it?

Answer: Use a Boolean variable and a conditional count expression, or separate environment configuration. Ensure the production input is set correctly and review the plan.

Q2. What risk exists when using count with a list of names?

Answer: Removing or reordering list items can change index-to-resource associations. Terraform may plan replacements or other changes because resources are identified by numeric indexes.

Q3. Can you use count and for_each in the same resource block?

Answer: No. A single resource or module block cannot use both meta-arguments to control its instances.

8. for_each vs count

8.1 Definition — What is the difference?

Both for_each and count can create multiple resource or module instances. They differ in how instances are identified and in the types of input they accept.

8.2 Purpose — When should each be used?

Use count when you need a number of similar instances or a simple optional resource. Use for_each when each instance has its own meaningful identity, especially when managing named resources from a map or string set.

8.3 Working — Comparison

Feature	count	for_each
Input	Whole number	Map or set of strings
Instance identity	Numeric index	Map key or set member
Example address	aws_instance.app[0]	aws_instance.app["api"]
Good for	Fixed quantities, optional resources	Named resources and keyed collections
Typical risk	Index shifts after list changes	Key changes or removals can destroy/recreate instances
Both allowed together?	No	No

> **Kannada explanation:** Number ಆಧಾರದಲ್ಲಿ resources ಬೇಕಾದರೆ count ಸರಳ. ಹೆಸರು ಅಥವಾ key ಆಧಾರದಲ್ಲಿ resources manage ಮಾಡಬೇಕಾದರೆ for_each ಸಾಮಾನ್ಯವಾಗಿ ಉತ್ತಮ.

8.4 Practical Example — Choosing the correct approach

Use count for three identical worker instances:

```
resource "aws_instance" "worker" {
  count         = 3
  ami           = var.ami_id
  instance_type = "t3.micro"
}
```

Use for_each for named services:

```
variable "services" {
  type = map(string)

  default = {
    api    = "t3.small"
    worker = "t3.micro"
  }
}

resource "aws_instance" "service" {
  for_each      = var.services
  ami           = var.ami_id
  instance_type = each.value

  tags = {
    Name = each.key
  }
}
```

> **Kannada explanation:** ಇಲ್ಲಿ workerಗಳಿಗೆ ಸಂಖ್ಯೆ ಮುಖ್ಯ. API ಮತ್ತು worker resourcesಗೆ ಹೆಸರು ಮುಖ್ಯ. ಆದ್ದರಿಂದ ಮೊದಲ ಉದಾಹರಣೆಯಲ್ಲಿ count, ಎರಡನೇ ಉದಾಹರಣೆಯಲ್ಲಿ for_each ಬಳಸಲಾಗಿದೆ.

8.5 Scenario-based Interview Questions

Q1. You manage 20 EC2 instances identified by application names. Which meta-argument would you choose?

Answer: Usually for_each with a map keyed by application name, because the resource addresses remain tied to meaningful keys.

Q2. You need zero or one optional resource. Which is appropriate?

Answer: Conditional count is often straightforward, although a conditional map with for_each is also possible.

Q3. Why can changing a count list be risky?

Answer: Instances use numeric indexes. Removing an item can shift index associations and cause Terraform to propose unwanted changes. Review the plan and consider for_each for individually named resources.

9. Conditional Expressions and for Expressions

9.1 Definition — What are conditional and for expressions?

A conditional expression chooses one value from two alternatives based on a condition.

Syntax:

```
condition ? true_value : false_value
```

A for expression transforms a collection into a new list, set, or map by evaluating each element.

> **Kannada explanation:** Conditional expression ಅಂದ್ರೆ condition true ಅಥವಾ false ಆಗಿರುವುದರ ಮೇಲೆ value ಆಯ್ಕೆ ಮಾಡುವುದು. for expression ಅಂದ್ರೆ list ಅಥವಾ mapನಲ್ಲಿರುವ values ಮೇಲೆ loop ಮಾಡಿ ಹೊಸ collection ಸೃಷ್ಟಿಸುವುದು.

9.2 Purpose — Why do we use them?

- Make environment-specific configurations.
- Select instance types based on conditions.
- Filter or transform input collections.
- Generate tags, names, and resource configuration dynamically.
- Avoid repetitive hardcoded values.

> **Kannada explanation:** Devಗೆ ಒಂದು value, productionಗೆ ಬೇರೆ value ಬೇಕಾದಾಗ conditional ಉಪಯೋಗವಾಗುತ್ತದೆ. Listನಿಂದ ಬೇಕಾದ values ಮಾತ್ರ ತೆಗೆದುಕೊಳ್ಳಲು ಅಥವಾ values ಬದಲಾಯಿಸಲು for expression ಸಹಾಯ ಮಾಡುತ್ತದೆ.

9.3 Working — How do they work?

Terraform evaluates the condition or iterates through the input collection during expression evaluation.

Conditional example:

```
var.environment == "prod" ? "t3.medium" : "t3.micro"
```

for expression example:

```
[for name in var.server_names : upper(name)]
```

If server_names contains ["api", "worker"], the result is ["API", "WORKER"].

9.4 Practical Example — Environment-specific instance type

```
variable "environment" {
  type = string
}

locals {
  instance_type = var.environment == "prod" ? "t3.medium" : "t3.micro"
}

resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = local.instance_type

  tags = {
    Environment = var.environment
  }
}
```

Filter enabled applications using a for expression:

```
variable "applications" {
  type = map(bool)

  default = {
    api    = true
    worker = true
    test   = false
  }
}

locals {
  enabled_applications = [
    for name, enabled in var.applications : name
    if enabled
  ]
}
```

The result is ["api", "worker"].

> **Kannada explanation:** Production environment ಇದ್ದರೆ t3.medium, ಇಲ್ಲದಿದ್ದರೆ t3.micro ಆಯ್ಕೆ ಆಗುತ್ತದೆ. ಎರಡನೇ ಉದಾಹರಣೆಯಲ್ಲಿ true ಆಗಿರುವ applications ಮಾತ್ರ ಹೊಸ listಗೆ ಸೇರುತ್ತವೆ.

9.5 Scenario-based Interview Questions

Q1. How would you select a larger EC2 instance type only for production?

Answer: Use a conditional expression or a validated environment-to-instance-type map. Review the resulting plan before applying.

Q2. How do you filter a list to include only enabled applications?

Answer: Use a for expression with an if clause.

Q3. What is the difference between a for expression and for_each?

Answer: A for expression transforms data into a new collection. for_each creates resource or module instances from a map or set of strings.

10. depends_on, lifecycle, and Advanced Resource Management

10.1 Definition — What are depends_on and lifecycle?

depends_on explicitly declares a dependency between Terraform resources or modules when Terraform cannot infer the required ordering from references alone.

lifecycle is a resource meta-argument block that controls certain aspects of resource creation, update, and destruction behavior.

Important lifecycle settings include:

- create_before_destroy
- prevent_destroy
- ignore_changes
- replace_triggered_by

> **Kannada explanation:** depends_on ಅಂದ್ರೆ ಯಾವ resource ಮೊದಲು create ಆಗಬೇಕು ಅನ್ನೋದನ್ನು Terraformಗೆ ಸ್ಪಷ್ಟವಾಗಿ ಹೇಳುವುದು. lifecycle ಅಂದ್ರೆ resource update ಅಥವಾ destroy ಆಗುವಾಗ Terraform ಹೇಗೆ ವರ್ತಿಸಬೇಕು ಅನ್ನೋದನ್ನು ನಿಯಂತ್ರಿಸುವುದು.

10.2 Purpose — Why do we use them?

- Express dependencies that are not clear from direct references.
- Reduce disruption during replacements.
- Prevent accidental destruction of critical resources.
- Ignore selected attributes that are intentionally managed elsewhere.
- Trigger replacement when a related resource or value changes.

> **Kannada explanation:** ಕೆಲವು resources ನಡುವೆ direct reference ಇಲ್ಲದಿದ್ದರೂ ಒಂದು resource ಇನ್ನೊಂದರ ನಂತರ create ಆಗಬೇಕು. ಅಲ್ಲಿ depends_on ಸಹಾಯ ಮಾಡುತ್ತದೆ. Critical resource ಅನ್ನು ತಪ್ಪಾಗಿ destroy ಆಗದಂತೆ ತಡೆಯಲು prevent_destroy ಬಳಸಬಹುದು.

10.3 Working — How do they work?

Terraform normally builds a dependency graph from resource references. It uses this graph to determine operation order.

Explicit depends_on adds a dependency to that graph. Lifecycle settings influence Terraform's planned actions, but they do not remove the need to inspect plans and manage dependencies carefully.

10.4 Practical Example — EC2 replacement and dependencies

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = var.instance_type

  lifecycle {
    create_before_destroy = true
  }
}
```

This asks Terraform to create the replacement before destroying the old instance when replacement is required and the provider and infrastructure permit that sequence.

For a critical resource:

```
resource "aws_s3_bucket" "critical_data" {
  bucket = var.bucket_name

  lifecycle {
    prevent_destroy = true
  }
}
```

prevent_destroy can block a planned destroy while the lifecycle rule remains in the configuration. It is not a backup, and it does not protect against every out-of-band deletion or every way the configuration may be changed.

Explicit dependency example:

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  depends_on = [
    aws_iam_role_policy_attachment.app
  ]
}
```

This illustrates an explicit ordering dependency on the policy attachment. In a real configuration, use the actual resource names and references.

> **Kannada explanation:** create_before_destroy replacement ಸಮಯದಲ್ಲಿ ಮೊದಲು ಹೊಸ resource create ಮಾಡಲು ಪ್ರಯತ್ನಿಸುತ್ತದೆ. prevent_destroy configurationನಿಂದ resource ತಪ್ಪಾಗಿ delete ಆಗುವುದನ್ನು ತಡೆಯಬಹುದು. depends_on ನಿರ್ದಿಷ್ಟ dependency order ಕೊಡುತ್ತದೆ.

10.5 Scenario-based Interview Questions

Q1. A critical S3 bucket must not be accidentally destroyed during a routine Terraform apply. What would you do?

Answer: Consider prevent_destroy, IAM restrictions, S3 versioning, backups, and production approval controls. prevent_destroy is one safeguard, not a complete data-protection strategy.

Q2. When should you use depends_on?

Answer: Use it when there is a genuine dependency Terraform cannot infer from normal references. Avoid adding it everywhere because unnecessary dependencies can make plans more conservative.

Q3. What does ignore_changes do?

Answer: It tells Terraform to ignore changes to specified attributes when planning updates. Use it only when the ignored changes are intentional; otherwise, configuration drift can go unnoticed.

Q4. Does create_before_destroy guarantee zero downtime?

Answer: No. It affects replacement ordering, but capacity limits, unique names, load balancer behavior, health checks, and application design determine whether downtime can actually be avoided.

11. Module Composition and Environment Design

11.1 Definition — What is module composition?

Module composition means combining smaller Terraform modules to build a complete infrastructure architecture.

For example, a root module can call networking, security group, IAM, EKS, and monitoring modules, passing outputs from one module into another.

> **Kannada explanation:** ದೊಡ್ಡ infrastructure ಅನ್ನು ಒಂದೇ ದೊಡ್ಡ Terraform fileನಲ್ಲಿ ಬರೆಯುವ ಬದಲು networking, IAM, EC2 ಅಥವಾ EKS ಅಂತ ಪ್ರತ್ಯೇಕ modules ಮಾಡಿ, ಅವುಗಳನ್ನು ಒಟ್ಟಿಗೆ connect ಮಾಡುವುದೇ module composition.

11.2 Purpose — Why do we use it?

- Organize infrastructure by responsibility.
- Reuse tested infrastructure components.
- Define clear interfaces between components.
- Simplify team ownership and code reviews.
- Make large environments easier to understand.

> **Kannada explanation:** Networking team ಒಂದು module maintain ಮಾಡಬಹುದು, EKS team ಮತ್ತೊಂದು module maintain ಮಾಡಬಹುದು. Modules outputs ಮೂಲಕ connect ಆಗುವುದರಿಂದ project maintain ಮಾಡುವುದು ಸುಲಭವಾಗುತ್ತದೆ.

11.3 Working — How does module composition work?

A common pattern is:

1. The networking module creates a VPC and subnets.
2. It exposes subnet IDs through outputs.

3. The security module configures the required network access.
4. The EKS module receives subnet IDs and other required inputs.
5. The root module combines the outputs and inputs.
6. Terraform builds a dependency graph and plans the required changes.

11.4 Practical Example — Compose an AWS EKS architecture

Example root main.tf:

```
module "network" {
  source = "./modules/network"

  vpc_cidr = var.vpc_cidr
}

module "eks" {
  source = "./modules/eks"

  cluster_name = var.cluster_name
  subnet_ids   = module.network.private_subnet_ids
}
```

The network module must define the private_subnet_ids output, and the EKS module must declare a compatible subnet_ids input.

> **Kannada explanation:** Network module private subnet IDs ಅನ್ನು output ಮಾಡುತ್ತದೆ. EKS module ಆ IDs ಅನ್ನು input ಆಗಿ ಪಡೆಯುತ್ತದೆ. ಇದರಿಂದ EKS cluster ಸರಿಯಾದ subnets ಬಳಸುವಂತೆ modules connect ಆಗುತ್ತವೆ.

11.5 Scenario-based Interview Questions

Q1. You need to create an EKS cluster inside a VPC created by another module. How would you connect them?

Answer: Expose the subnet IDs from the networking module through outputs, then pass those outputs as inputs to the EKS module.

Q2. Why should module interfaces be kept simple?

Answer: Clear inputs and outputs reduce coupling, simplify testing, and make modules easier to reuse and upgrade.

Q3. Should dev and production always share the same root module?

Answer: Not always. Similar environments may share reusable child modules, but separate root configurations or accounts may be better when access, state, networking, or release controls differ significantly.

12. Advanced Terraform Best Practices and Production Scenarios

12.1 Definition — What are advanced Terraform patterns?

Advanced Terraform patterns are design practices that help manage infrastructure safely and consistently at scale. They include typed variables, input validation, module versioning, environment separation, state management, lifecycle controls, and automated plan review.

> **Kannada explanation:** Advanced patterns ಅಂದ್ರೆ ಕೇವಲ resources create ಮಾಡುವುದಲ್ಲ. ದೊಡ್ಡ projectನಲ್ಲಿ code maintainable ಆಗಿರಬೇಕು, production ಸುರಕ್ಷಿತವಾಗಿರಬೇಕು ಮತ್ತು ತಪ್ಪು changes ಆಗದಂತೆ process ಇರಬೇಕು.

12.2 Purpose — Why do we use them?

- Improve security and consistency.
- Reduce unexpected production changes.
- Support automated CI/CD pipelines.
- Make infrastructure easier to review and troubleshoot.
- Improve repeatability and maintainability.

12.3 Working — How do they work?

A mature Terraform workflow typically includes:

1. Store infrastructure code in Git.
2. Use validated input variables and reusable modules.
3. Pin Terraform, provider, and module versions where appropriate.
4. Keep state in a secure remote backend for team environments.
5. Run formatting, validation, and automated checks.
6. Generate and review a Terraform plan.
7. Apply changes through approved CI/CD stages.
8. Monitor infrastructure and handle drift through a controlled workflow.

> **Kannada explanation:** Developer code Gitಗೆ push ಮಾಡುತ್ತಾರೆ. Pipeline formatting, validation ಮತ್ತು plan run ಮಾಡುತ್ತದೆ. Plan review ಅಥವಾ approval ಆದ ಮೇಲೆ apply ಮಾಡಲಾಗುತ್ತದೆ. ಹೀಗೆ ಮಾಡಿದರೆ productionನಲ್ಲಿ ತಪ್ಪು changes ಆಗುವ ಅಪಾಯ ಕಡಿಮೆಯಾಗುತ್ತದೆ.

12.4 Practical Example — Terraform checks in Jenkins

A simple pipeline stage sequence could be:

```
stage('Terraform Validate') {
    steps {
        bat 'terraform fmt -check -recursive'
        bat 'terraform init -input=false'
        bat 'terraform validate'
    }
}

stage('Terraform Plan') {
    steps {
        bat 'terraform plan -input=false -out=tfplan'
    }
}
```

This is an illustrative Jenkins pipeline fragment, not a complete production-ready pipeline. It assumes Terraform is installed, AWS credentials are configured securely, the correct working directory is selected, and the backend is initialized properly.

For production:

- Protect credentials using Jenkins credentials or an approved identity mechanism.
- Do not commit AWS access keys or sensitive state files to Git.
- Protect plan files because they may contain sensitive values.
- Require suitable approvals before applying production changes.
- Use a remote backend with appropriate access controls and state recovery measures.
- Avoid running competing Terraform operations against the same state.

12.5 Scenario-based Interview Questions

Q1. A Jenkins pipeline proposes destroying an EKS cluster unexpectedly. What would you do?

Answer: Follow these steps:

1. Do not approve or apply the plan.
2. Review the complete plan for replacements and deletions.
3. Confirm the correct Git branch, workspace, variable files, AWS account, and region.
4. Check the backend configuration and state consistency.
5. Investigate changes to resource names, keys, module versions, and configuration.
6. Correct the cause, regenerate the plan, and obtain the required approval.

> **Kannada:** EKS cluster destroy ಆಗುತ್ತದೆ ಎಂದು plan ತೋರಿಸಿದರೆ ತಕ್ಷಣ apply ಮಾಡಬಾರದು. ಮೊದಲು account, region, workspace, state ಮತ್ತು configuration ಸರಿಯಾಗಿದೆಯೇ ಎಂದು ಪರಿಶೀಲಿಸಬೇಕು.

Q2. Your for_each key changes from api to backend. What could happen?

Answer: Terraform treats these as different resource addresses. It may plan to destroy the old instance and create a new one. If the underlying resource should remain the same, consider a carefully reviewed terraform state mv or moved block migration, as appropriate for the resource and Terraform version.

Q3. Your team wants to upgrade a shared module used by many applications. How would you proceed?

Answer: Release a new version, test it in a lower environment, review plan changes for representative consumers, and roll out the upgrade in stages with appropriate approvals.

Q4. How would you reduce Terraform code duplication in an AWS EKS project?

Answer: Create reusable modules for networking, IAM roles and policies, security groups, EKS clusters, and node groups. Use typed inputs and outputs, and keep environment-specific values in controlled configuration files.

Q5. How do you keep production infrastructure safe in a Terraform CI/CD pipeline?

Answer: Use secure credentials, a remote state backend, appropriate locking, plan review, controlled approvals, least-privilege IAM permissions, and reliable state backups.

13. Quick Revision — Important Commands and Concepts

13.1 Terraform commands

Command	Purpose
terraform init	Initialize the configuration and download required providers and modules
terraform fmt -recursive	Format Terraform files
terraform validate	Validate configuration syntax and internal consistency
terraform plan	Preview proposed infrastructure changes
terraform apply	Apply the approved or reviewed infrastructure changes
terraform workspace list	List available workspaces
terraform workspace new dev	Create a workspace named dev
terraform workspace select dev	Select the dev workspace
terraform workspace show	Display the current workspace

13.2 Advanced Terraform cheat sheet

Requirement	Recommended feature
Reuse EC2 creation code	Modules
Pass values into a module	Input variables
Return a resource ID from a module	Outputs
Manage multiple named resources	for_each
Create a fixed number of resources	count
Generate repeated nested blocks	dynamic
Choose values based on conditions	Conditional expressions
Transform or filter collections	for expressions
Declare an otherwise implicit dependency	depends_on
Control replacement and destruction behavior	lifecycle
Maintain multiple state instances	Workspaces
Reuse a module across projects	Shared or versioned modules

13.3 Common interview traps

- Modules do not automatically create infrastructure. They must be called by an active configuration and applied.
- for_each and count are not interchangeable in every situation. Choose based on whether stable named identities or numeric quantities are more appropriate.
- Dynamic blocks do not create independent resources. They generate nested blocks.
- Workspaces do not create separate AWS accounts. Account and permission isolation must be configured separately.
- prevent_destroy is not a backup. Use backups and recovery controls for critical data.
- create_before_destroy does not guarantee zero downtime. Infrastructure and application design also matter.
- Changing a for_each key or count index can cause resource changes. Inspect the plan before applying.

14. Practice Checklist

Use this checklist to check your understanding.

Your learning progress: 0 of 12

- [ ] I can create a reusable EC2 module with inputs and outputs.
- [ ] I can call the same module from different environments.
- [ ] I understand root modules and child modules.
- [ ] I can explain Terraform workspace state separation.
- [ ] I can use a dynamic block to generate security group rules.
- [ ] I can create multiple resources using for_each.
- [ ] I can create a fixed number of resources using count.
- [ ] I can explain the differences between count and for_each.
- [ ] I can use conditional expressions and for expressions.
- [ ] I understand depends_on and lifecycle rules.
- [ ] I can connect networking and EKS modules using outputs.
- [ ] I can troubleshoot an unexpected destroy plan in Jenkins.

Final interview takeaway

For advanced Terraform interviews, focus especially on modules, for_each vs count, dynamic blocks, workspace limitations, module outputs, and lifecycle behavior. Be prepared to explain not only the syntax but also how changes affect resource addresses, Terraform state, and production deployments.

> For your AWS Terraform and Jenkins projects, a strong practical exercise is to build reusable networking and EC2/EKS modules, connect them through inputs and outputs, run a plan in Jenkins, and safely review the proposed changes before applying them.